

CSS Advanced Selectors

Pseudo-Classes Guide

Style elements based on their state and position

Easy English • Practical Examples • Ready to Use

Contents

1	Introduction to Pseudo-Classes	2
1.1	What are Pseudo-Classes?	2
1.2	Why Use Pseudo-Classes?	2
1.3	Pseudo-Classes vs Pseudo-Elements	2
2	User Action Pseudo-Classes	3
2.1	1. :hover	3
	2.1.1 Explanation	3
	2.1.2 Example	3
2.2	2. :active	5
	2.2.1 Explanation	5
	2.2.2 Example	5
2.3	3. :focus	6
	2.3.1 Explanation	6
	2.3.2 Example	6
2.4	4. :focus-visible	7
	2.4.1 Explanation	7
	2.4.2 Example	7
2.5	5. :visited	8
	2.5.1 Explanation	8
	2.5.2 Example	8
3	Form Pseudo-Classes	9
3.1	1. :checked	9
	3.1.1 Explanation	9
	3.1.2 Example	9
3.2	2. :disabled	11
	3.2.1 Explanation	11
	3.2.2 Example	11
3.3	3. :enabled	12
	3.3.1 Explanation	12
	3.3.2 Example	12
3.4	4. :required and :optional	13
	3.4.1 Explanation	13
	3.4.2 Example	13
3.5	5. :valid and :invalid	14
	3.5.1 Explanation	14

3.5.2	Example	14
4	Structural Pseudo-Classes	15
4.1	1. :first-child	15
4.1.1	Explanation	15
4.1.2	Example	15
4.2	2. :last-child	16
4.2.1	Explanation	16
4.2.2	Example	16
4.3	3. :nth-child(n)	17
4.3.1	Explanation	17
4.3.2	Examples	17
4.4	4. :nth-last-child(n)	19
4.4.1	Explanation	19
4.4.2	Example	19
4.5	5. :only-child	20
4.5.1	Explanation	20
4.5.2	Example	20
4.6	6. :first-of-type and :last-of-type	21
4.6.1	Explanation	21
4.6.2	Example	21
4.7	7. :nth-of-type(n)	22
4.7.1	Explanation	22
4.7.2	Example	22
5	State Pseudo-Classes	23
5.1	1. :empty	23
5.1.1	Explanation	23
5.1.2	Example	23
5.2	2. :root	24
5.2.1	Explanation	24
5.2.2	Example	24
5.3	3. :target	25
5.3.1	Explanation	25
5.3.2	Example	25
5.4	4. :is() and :where()	26
5.4.1	Explanation	26
5.4.2	Example	26
6	Practical Examples	27
6.1	Example 1: Interactive Navigation Menu	27
6.2	Example 2: Table with Striped Rows	28
6.3	Example 3: Form Validation Styling	29
6.4	Example 4: List with First/Last Styling	30
7	Quick Reference	31
7.1	All Pseudo-Classes Summary	31

8	Tips and Best Practices	32
8.1	Tip 1: Always Include Focus Styles	32
8.2	Tip 2: Use :hover with :active	32
8.3	Tip 3: nth-child() Formulas	32
8.4	Tip 4: Use :is() for Cleaner Code	33
8.5	Tip 5: Understand Specificity with Pseudo-Classes	33
9	Common Mistakes to Avoid	34
9.1	Mistake 1: Forgetting :focus Styles	34
9.2	Mistake 2: Using nth-child() Incorrectly	34
9.3	Mistake 3: Confusing :nth-child() with :nth-of-type()	34
9.4	Mistake 4: Over-using :important	34
9.5	Mistake 5: Security Issue with :visited	35
10	Summary	36
11	Practice Exercises	37
11.1	Exercise 1: Interactive Button	37
11.2	Exercise 2: Form Validation	37
11.3	Exercise 3: Table Striping	37
11.4	Exercise 4: Navigation Menu	37
11.5	Exercise 5: Advanced nth-child()	38

Chapter 1

Introduction to Pseudo-Classes

1.1 What are Pseudo-Classes?

Pseudo-classes are CSS selectors that target elements based on their state or position in the document. They allow you to style elements dynamically without adding extra classes to your HTML. A pseudo-class starts with a single colon (:) followed by the pseudo-class name.

Think of it as: "Style an element when it's in a specific state or location."

Key Concept

Pseudo-classes let you style elements based on their state (like hovering, focusing, being active) or their position in the HTML structure (like first child, last child, nth child).

1.2 Why Use Pseudo-Classes?

- Create interactive effects without JavaScript
- Style elements based on their state
- Target specific elements by position
- Make your CSS more dynamic and responsive
- Improve user experience with visual feedback
- Reduce the need for extra HTML classes

1.3 Pseudo-Classes vs Pseudo-Elements

- **Pseudo-Classes (:):** Style elements based on state or position
- **Pseudo-Elements (::):** Create virtual elements or style specific parts

This guide focuses on pseudo-classes only.

Chapter 2

User Action Pseudo-Classes

2.1 1. :hover

2.1.1 Explanation

The `:hover` pseudo-class applies styles when a user hovers their mouse over an element. It's one of the most commonly used pseudo-classes.

In simple words: "Style an element when someone moves their mouse over it"

2.1.2 Example

CSS Code

```
a {  
    color: blue;  
    text-decoration: none;  
}  
  
a:hover {  
    color: red;  
    text-decoration: underline;  
}
```

HTML:

```
<a href="page.html">Hover over me</a>
```

Result: The link is blue without underline by default. When you hover over it, it becomes red with an underline.

Button hover example:

CSS Code

```
button {  
    background-color: blue;  
    color: white;  
    padding: 10px 20px;  
    cursor: pointer;  
}  
  
button:hover {  
    background-color: darkblue;  
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);  
}
```

2.2 2. :active

2.2.1 Explanation

The `:active` pseudo-class applies styles while an element is being activated (like while a mouse button is held down on a link or button).

In simple words: "Style an element while it's being clicked"

2.2.2 Example

CSS Code

```
button {  
    background-color: blue;  
    padding: 10px 20px;  
}  
  
button:active {  
    background-color: darkblue;  
    transform: scale(0.98);  
}
```

HTML:

```
<button>Click me</button>
```

Result: When you click and hold the button, it becomes darker and slightly smaller.

2.3 3. :focus

2.3.1 Explanation

The `:focus` pseudo-class applies styles when an element receives keyboard focus. This is commonly used on form inputs, buttons, and links when they're tabbed into.

In simple words: "Style an element when it has focus (via keyboard or mouse click)"

2.3.2 Example

CSS Code

```
input {  
  padding: 8px;  
  border: 2px solid #ccc;  
}  
  
input:focus {  
  outline: none;  
  border-color: blue;  
  box-shadow: 0 0 5px rgba(0, 0, 255, 0.3);  
}
```

HTML:

```
<input type="text" placeholder="Click or tab here">
```

Result: When the input is focused (clicked or tabbed into), the border turns blue and a subtle glow appears.

Accessibility Tip

`:focus` is important for keyboard navigation. Always ensure focusable elements have visible focus styles for accessibility.

2.4 4. :focus-visible

2.4.1 Explanation

The `:focus-visible` pseudo-class applies styles only when an element receives focus via keyboard (not mouse click). This helps distinguish between keyboard and mouse focus.

In simple words: "Style an element only when focused by keyboard"

2.4.2 Example

CSS Code

```
button {  
    padding: 10px 20px;  
    border: 2px solid transparent;  
}  
  
button:focus-visible {  
    outline: 3px solid blue;  
    outline-offset: 2px;  
}
```

Result: The button shows an outline only when focused via keyboard, not when clicked with the mouse.

2.5 5. :visited

2.5.1 Explanation

The `:visited` pseudo-class applies styles to links that the user has already visited. For security reasons, only a few CSS properties can be used with `:visited`.

In simple words: "Style links that have already been clicked"

2.5.2 Example

CSS Code

```
a {  
    color: blue;  
}  
  
a:visited {  
    color: purple;  
}
```

HTML:

```
<a href="https://google.com">Google</a>  
<a href="https://visited-site.com">Visited Site</a>
```

Result: Unvisited links appear blue. Links you've already visited appear purple.

Important Note

For security reasons, only limited CSS properties work with `:visited`: color, background-color, border-color, and outline-color.

Chapter 3

Form Pseudo-Classes

3.1 1. :checked

3.1.1 Explanation

The `:checked` pseudo-class applies styles to checkboxes and radio buttons that are checked by the user.

In simple words: "Style checkboxes and radio buttons when they're checked"

3.1.2 Example

CSS Code

```
input[type="checkbox"] {  
    width: 20px;  
    height: 20px;  
    cursor: pointer;  
}  
  
input[type="checkbox"]:checked {  
    accent-color: green;  
}  
  
label {  
    margin-left: 8px;  
}  
  
input[type="checkbox"]:checked + label {  
    font-weight: bold;  
    color: green;  
}
```

HTML:

```
<input type="checkbox" id="agree">  
<label for="agree">I agree to the terms</label>
```

Result: When checked, the checkbox turns green and the label becomes bold and green.

3.2 2. :disabled

3.2.1 Explanation

The `:disabled` pseudo-class applies styles to form elements that are disabled.

In simple words: "Style form elements that are disabled and can't be used"

3.2.2 Example

CSS Code

```
button {  
    padding: 10px 20px;  
    background-color: blue;  
    color: white;  
    cursor: pointer;  
}  
  
button:disabled {  
    background-color: #ccc;  
    cursor: not-allowed;  
    opacity: 0.6;  
}
```

HTML:

```
<button>Active Button</button>  
<button disabled>Disabled Button</button>
```

Result: The disabled button appears grayed out and the cursor changes to "not-allowed".

3.3 3. :enabled

3.3.1 Explanation

The `:enabled` pseudo-class applies styles to form elements that are enabled (the opposite of `:disabled`).

In simple words: "Style form elements that are active and can be used"

3.3.2 Example

CSS Code

```
input:enabled {  
    border: 2px solid green;  
    background-color: #f0f9ff;  
}  
  
input:disabled {  
    border: 2px solid red;  
    background-color: #f0f0f0;  
}
```

3.4 4. :required and :optional

3.4.1 Explanation

The `:required` pseudo-class targets form inputs marked as required. The `:optional` pseudo-class targets optional form inputs.

In simple words: "Style required fields differently from optional fields"

3.4.2 Example

CSS Code

```
input:required {  
    border: 2px solid orange;  
}  
  
input:required::after {  
    content: " *";  
    color: red;  
}  
  
input:optional {  
    border: 2px solid green;  
}
```

HTML:

```
<input type="text" required>  
<input type="text" optional>
```


3.5 5. :valid and :invalid

3.5.1 Explanation

The `:valid` pseudo-class targets form inputs with valid values. The `:invalid` pseudo-class targets inputs with invalid values.

In simple words: "Style form inputs based on whether their value is valid"

3.5.2 Example

CSS Code

```
input[type="email"]:valid {  
    border-color: green;  
    background-color: #f0fff0;  
}  
  
input[type="email"]:invalid {  
    border-color: red;  
    background-color: #fff0f0;  
}
```

HTML:

```
<input type="email" value="valid@email.com">  
<input type="email" value="invalid-email">
```

Result: Valid email inputs get a green border. Invalid emails get a red border.

Chapter 4

Structural Pseudo-Classes

4.1 1. :first-child

4.1.1 Explanation

The `:first-child` pseudo-class targets elements that are the first child of their parent.

In simple words: "Style the first element inside a parent"

4.1.2 Example

CSS Code

```
li:first-child {  
    font-weight: bold;  
    color: blue;  
}
```

HTML:

```
<ul>  
  <li>First Item (bold and blue)</li>  
  <li>Second Item</li>  
  <li>Third Item</li>  
</ul>
```

Result: Only the first list item becomes bold and blue.

4.2 2. :last-child

4.2.1 Explanation

The `:last-child` pseudo-class targets elements that are the last child of their parent.

In simple words: "Style the last element inside a parent"

4.2.2 Example

CSS Code

```
li:last-child {  
    border-bottom: none;  
}  
  
p:last-child {  
    margin-bottom: 0;  
}
```

HTML:

```
<ul>  
  <li>First Item</li>  
  <li>Second Item</li>  
  <li>Last Item (no border)</li>  
</ul>
```

4.3 3. :nth-child(n)

4.3.1 Explanation

The `:nth-child(n)` pseudo-class targets elements based on their position. You can use numbers, keywords, or formulas.

In simple words: "Style specific elements by their position number"

4.3.2 Examples

CSS Code - Specific Position

```
/* Select the 3rd item */
li:nth-child(3) {
    color: red;
}

/* Select the 5th item */
li:nth-child(5) {
    background-color: yellow;
}
```

CSS Code - Even/Odd

```
/* Select odd items */
tr:nth-child(odd) {
    background-color: #f0f0f0;
}

/* Select even items */
tr:nth-child(even) {
    background-color: white;
}
```

CSS Code - Formulas

```
/* Every 2nd item starting from item 1 */
li:nth-child(2n+1) {
    color: blue;
}

/* Every 3rd item */
li:nth-child(3n) {
    margin-bottom: 20px;
}

/* Every 4th item starting from item 2 */
li:nth-child(4n+2) {
    font-weight: bold;
}
```

4.4 4. :nth-last-child(n)

4.4.1 Explanation

The `:nth-last-child(n)` pseudo-class is like `:nth-child(n)` but counts from the end instead of the beginning.

In simple words: "Style elements by their position counting from the end"

4.4.2 Example

CSS Code

```
/* Last item */
li:nth-last-child(1) {
    color: red;
}

/* Second to last item */
li:nth-last-child(2) {
    color: orange;
}
```

4.5 5. :only-child

4.5.1 Explanation

The :only-child pseudo-class targets elements that are the only child of their parent.

In simple words: "Style elements that are the only child"

4.5.2 Example

CSS Code

```
p:only-child {  
    margin: 0;  
}
```

HTML:

```
<div>  
  <p>This is the only child</p>  
</div>  
  
<div>  
  <p>First paragraph</p>  
  <p>Second paragraph</p>  
</div>
```

Result: Only the first paragraph (which is an only child) gets the styling.

4.6 6. :first-of-type and :last-of-type

4.6.1 Explanation

The `:first-of-type` targets the first element of a specific type. The `:last-of-type` targets the last element of a specific type.

In simple words: "Style the first/last element of a specific tag type"

4.6.2 Example

CSS Code

```
p:first-of-type {  
    font-weight: bold;  
}  
  
p:last-of-type {  
    margin-bottom: 0;  
}
```

HTML:

```
<article>  
  <h2>Title</h2>  
  <p>First paragraph (bold)</p>  
  <p>Second paragraph</p>  
  <p>Last paragraph (no margin)</p>  
</article>
```

Result: The first `<p>` becomes bold, and the last `<p>` has no margin, regardless of other elements.

4.7 7. :nth-of-type(n)

4.7.1 Explanation

The `:nth-of-type(n)` pseudo-class targets elements by position, but only counts elements of the same type.

In simple words: "Style specific positions of a particular element type"

4.7.2 Example

CSS Code

```
/* Every 2nd paragraph */
p:nth-of-type(2n) {
    background-color: #f0f0f0;
}

/* 3rd paragraph */
p:nth-of-type(3) {
    color: red;
}
```

HTML:

```
<article>
  <h2>Title</h2>
  <p>Paragraph 1</p>
  <p>Paragraph 2 (gray background)</p>
  <p>Paragraph 3 (red)</p>
</article>
```

Chapter 5

State Pseudo-Classes

5.1 1. :empty

5.1.1 Explanation

The `:empty` pseudo-class targets elements that have no children at all (not even whitespace).

In simple words: "Style elements that are completely empty"

5.1.2 Example

CSS Code

```
div:empty {  
    display: none;  
}  
  
p:empty {  
    border: 2px dashed red;  
    min-height: 50px;  
}
```

HTML:

```
<div></div>  
<p></p>  
<p>Has content</p>
```

Result: Empty elements get special styling to make them visible during development.

5.2 2. :root

5.2.1 Explanation

The `:root` pseudo-class targets the root element of the document (the `<html>` tag in most cases).

In simple words: "Style the root element or define global CSS variables"

5.2.2 Example

CSS Code

```
:root {  
  --primary-color: blue;  
  --secondary-color: green;  
  --font-size: 16px;  
}  
  
body {  
  background-color: var(--primary-color);  
  font-size: var(--font-size);  
}  
  
button {  
  background-color: var(--secondary-color);  
}
```

Result: CSS variables defined in `:root` can be used throughout the entire stylesheet.

5.3 3. :target

5.3.1 Explanation

The `:target` pseudo-class applies styles to the element that is the target of an anchor link.

In simple words: "Style an element when it's linked to via a URL fragment"

5.3.2 Example

CSS Code

```
:target {  
    background-color: yellow;  
    border: 2px solid red;  
}
```

HTML:

```
<a href="#section1">Go to Section 1</a>  
  
<section id="section1">  
    <h2>Section 1</h2>  
    <p>This section is highlighted when linked</p>  
</section>
```

Result: When you click the link or visit the URL with `#section1`, the section gets highlighted with yellow background and red border.

5.4 4. :is() and :where()

5.4.1 Explanation

The `:is()` and `:where()` pseudo-classes let you apply the same styles to multiple selectors with less code. The difference is in specificity handling.

In simple words: "Apply styles to multiple selectors at once"

5.4.2 Example

Without :is() - Verbose

```
h1, h2, h3, h4, h5, h6 {  
    color: blue;  
    margin-bottom: 20px;  
}
```

With :is() - Cleaner

```
:is(h1, h2, h3, h4, h5, h6) {  
    color: blue;  
    margin-bottom: 20px;  
}
```

More complex example:

Complex Selector with :is()

```
/* Style all headings inside article or section */  
:is(article, section) :is(h1, h2, h3) {  
    color: blue;  
}
```

Chapter 6

Practical Examples

6.1 Example 1: Interactive Navigation Menu

HTML

```
<nav>
  <a href="#home">Home</a>
  <a href="#about">About</a>
  <a href="#contact">Contact</a>
</nav>
```

CSS with Pseudo-Classes

```
nav a {
  color: white;
  background-color: #333;
  padding: 12px 20px;
  text-decoration: none;
  transition: all 0.3s ease;
}

nav a:hover {
  background-color: blue;
  transform: translateY(-2px);
}

nav a:active {
  background-color: darkblue;
}

nav a:focus {
  outline: 3px solid yellow;
}
```

6.2 Example 2: Table with Striped Rows

HTML

```
<table>
  <tr>
    <td>Row 1</td>
  </tr>
  <tr>
    <td>Row 2</td>
  </tr>
  <tr>
    <td>Row 3</td>
  </tr>
</table>
```

CSS with Pseudo-Classes

```
tr:nth-child(odd) {
  background-color: #f0f0f0;
}

tr:nth-child(even) {
  background-color: white;
}

tr:hover {
  background-color: #e0e0e0;
}
```

6.3 Example 3: Form Validation Styling

HTML

```
<form>
  <input type="email" required placeholder="Email">
  <input type="password" required placeholder="Password">
  <button type="submit">Login</button>
</form>
```

CSS with Pseudo-Classes

```
input {
  padding: 10px;
  margin: 10px 0;
  border: 2px solid #ccc;
}

input:focus {
  outline: none;
  border-color: blue;
  box-shadow: 0 0 5px rgba(0, 0, 255, 0.3);
}

input:valid {
  border-color: green;
  background-color: #f0fff0;
}

input:invalid {
  border-color: red;
  background-color: #fff0f0;
}

button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

button:enabled:hover {
  background-color: blue;
  color: white;
}
```


6.4 Example 4: List with First/Last Styling

HTML

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
  <li>Item 4</li>
</ul>
```

CSS with Pseudo-Classes

```
li {
  padding: 10px;
  border-bottom: 1px solid #ddd;
}

li:first-child {
  font-weight: bold;
  background-color: #e0e0ff;
}

li:last-child {
  border-bottom: none;
}

li:hover {
  background-color: #f0f0f0;
}
```

Chapter 7

Quick Reference

7.1 All Pseudo-Classes Summary

Pseudo-Class	What It Does	Example
:hover	Mouse over	a:hover
:active	Being clicked	button:active
:focus	Has focus	input:focus
:visited	Link visited	a:visited
:checked	Checkbox checked	input:checked
:disabled	Form disabled	button:disabled
:enabled	Form enabled	input:enabled
:valid	Valid form value	input:valid
:invalid	Invalid form value	input:invalid
:first-child	First child	li:first-child
:last-child	Last child	li:last-child
:nth-child(n)	Nth position	li:nth-child(2)
:empty	No content	div:empty
:root	Root element	:root
:target	Anchor target	:target

Chapter 8

Tips and Best Practices

8.1 Tip 1: Always Include Focus Styles

For accessibility, always include `:focus` or `:focus-visible` styles on interactive elements:

```
button {  
    padding: 10px 20px;  
}  
  
button:focus-visible {  
    outline: 3px solid blue;  
    outline-offset: 2px;  
}
```

8.2 Tip 2: Use `:hover` with `:active`

Combine `:hover` and `:active` for better user feedback:

```
button:hover {  
    background-color: lightblue;  
}  
  
button:active {  
    background-color: darkblue;  
}
```

8.3 Tip 3: `nth-child()` Formulas

Master `nth-child` formulas for advanced selections:

```
/* Every 2nd element */  
li:nth-child(2n) { }  
  
/* Every 3rd starting from 1st */  
li:nth-child(3n+1) { }  
  
/* First 3 elements */  
li:nth-child(-n+3) { }
```

```
/* All elements except first */  
li:nth-child(n+2) { }
```

8.4 Tip 4: Use :is() for Cleaner Code

Use :is() to reduce selector repetition:

```
/* Verbose */  
h1, h2, h3 { color: blue; }  
  
/* Clean */  
:is(h1, h2, h3) { color: blue; }
```

8.5 Tip 5: Understand Specificity with Pseudo-Classes

Pseudo-classes increase specificity. Arrange your CSS accordingly:

```
/* Lower specificity */  
a { color: blue; }  
  
/* Medium specificity */  
a:hover { color: red; }  
  
/* Override with important if needed */  
a:active { color: darkred !important; }
```

Chapter 9

Common Mistakes to Avoid

9.1 Mistake 1: Forgetting :focus Styles

```
/* WRONG - No focus style for accessibility */
input { border: 2px solid #ccc; }
input:hover { border-color: blue; }

/* RIGHT - Include focus style */
input { border: 2px solid #ccc; }
input:hover { border-color: blue; }
input:focus { outline: 3px solid blue; }
```

9.2 Mistake 2: Using nth-child() Incorrectly

```
/* WRONG - Counts all children, not just li */
li:nth-child(2) { }
```

```
/* BETTER - Use nth-of-type for type-specific */
li:nth-of-type(2) { }
```

9.3 Mistake 3: Confusing :nth-child() with :nth-of-type()

```
/* :nth-child(3) - 3rd child of ANY type */
p:nth-child(3) { }
```

```
/* :nth-of-type(3) - 3rd p element specifically */
p:nth-of-type(3) { }
```

9.4 Mistake 4: Over-using :important

```
/* WRONG - Unnecessary !important */
a:hover { color: red !important; }
```

```
/* RIGHT - Let CSS cascade work */  
a:hover { color: red; }
```

9.5 Mistake 5: Security Issue with :visited

```
/* WRONG - Limited properties work with :visited */  
a:visited { background: url(tracker.gif); }  
  
/* RIGHT - Use safe properties only */  
a:visited { color: purple; }
```

Chapter 10

Summary

What You Learned

5 Categories of Pseudo-Classes:

1. **User Actions:** `:hover`, `:active`, `:focus`, `:visited`
2. **Form States:** `:checked`, `:disabled`, `:enabled`, `:valid`, `:invalid`
3. **Structure:** `:first-child`, `:last-child`, `:nth-child()`, `:only-child`
4. **State:** `:empty`, `:root`, `:target`
5. **Logical:** `:is()`, `:where()`

Key takeaways:

- Pseudo-classes let you style elements based on their state or position
- Always include `:focus` styles for accessibility
- Use structural pseudo-classes to reduce CSS code
- Combine pseudo-classes with other selectors for powerful styling
- Master `nth-child()` formulas for advanced selections

Practice using these pseudo-classes and create more interactive, accessible websites!

Chapter 11

Practice Exercises

11.1 Exercise 1: Interactive Button

Create a button that changes style on:

- Hover
- Active (being clicked)
- Focus (keyboard focus)

11.2 Exercise 2: Form Validation

Create a form with:

- Email input that validates
- Required field styling
- Focus styling
- Valid/invalid styling

11.3 Exercise 3: Table Striping

Create a table with:

- Alternating row colors
- Hover effect on rows
- Different styling for first and last row

11.4 Exercise 4: Navigation Menu

Build a navigation with:

- Hover effects on links

- Focus states for accessibility
- Visited link styling
- Active link highlighting

11.5 Exercise 5: Advanced nth-child()

Create a list that shows:

- Every 2nd item in one color
- Every 3rd item in another color
- First item special styling
- Last item different styling

Complete these exercises to master pseudo-classes!